

Guía Integral del Proyecto: One-Time Vault (OTV) – Compartición de Secretos de Un Solo Uso (E2E)

Curso: Ética y Seguridad de los Datos · Grupo: 4 estudiantes de Ciencia de Datos · Duración sugerida: 1 semana intensiva (o 2 semanas light).

Resumen: Web app para compartir secretos mediante enlaces de un solo uso. El secreto se cifra en el navegador con WebCrypto; el servidor almacena únicamente el ciphertext y metadatos mínimos. Incluye extras de ciberseguridad: canary tokens, log de transparencia encadenado, borrado criptográfico, PoW anti-abuso y hardening del navegador.

Objetivos de aprendizaje y de impacto

- Aplicar principios de Privacy by Design: minimización, zero-knowledge, separación de poderes.
- Implementar cifrado E2E real en cliente (AES-GCM/ChaCha20-Poly1305) y ECDH (X25519) para intercambio de claves.
- Diseñar controles adicionales fuera de la rúbrica: canary tokens, transparencia verificable y borrado criptográfico.
- Entregar una demo convincente en 5 minutos y documentación auditable.

Arquitectura y componentes

Frontend (React/Vite): Web Crypto API (AES-GCM, HKDF, PBKDF Argon2id vía WASM), generación/almacenamiento efímero de claves, construcción de enlaces con fragmento `#key`, QR opcional, UI minimalista.

Backend (FastAPI o Node.js/Express): Endpoints `/secret` (POST), `/s/{id}` (GET/DELETE), `/log` (GET). Rate-limit, verificación de PoW, expiración y 'one-shot semantics'.

Base de datos: SQLite/Postgres. Tabla `secrets` (id, ciphertext, policy, created_at, expires_at, burned). Tabla `audit_log` con encadenamiento hash (prev_hash → hash).

Seguridad transversal: TLS, CSP estricta, HSTS, Referrer-Policy:no-referrer, Permissions-Policy mínima, SRI, backup cifrado de metadatos, runbook de incidentes.

Paso a paso (guía de implementación)

Día 1 – Setup y esqueleto

- Inicializar repos: frontend (React + Vite) y backend (FastAPI/Node).
- Configurar TLS local (mkcert/auto-firmado), linter y pruebas básicas.
- Modelo de datos mínimo y migración (secrets, audit_log).

Día 2 – Cifrado en cliente y creación de secretos

- Implementar generación de clave simétrica (AES-GCM 256) y IV aleatorio.
- Opción A (rápida): derivar clave con passphrase (Argon2id) + sal aleatoria.
- Opción B (wow): ECDH X25519 (claves efímeras) + HKDF para derivar K.
- Cifrar en cliente, POST de ciphertext + policy (nº lecturas=1, TTL).

- Construir URL con fragmento #key y flow de copiado seguro.

Día 3 – Lectura única y borrado criptográfico

- Endpoint GET /s/{id}: devuelve ciphertext solo si no ha sido leído/expirado.
- Cliente descifra con la clave del fragmento y muestra secreto una sola vez.
- Marcar como 'burned' y eliminar/invalidar wrap key (borrado criptográfico).
- Respuestas 404/410 controladas y mensajes UX claros.

Día 4 – Log de transparencia y canary tokens

- Implementar audit_log encadenado (hash(prev)+event → hash). Export JSON.
- Vista para verificar cadena (recalcular hash raíz).
- Canary: toggle que, al leer desde contexto sospechoso (geo/IP/user-agent), registra alerta y etiqueta el evento.

Día 5 – Hardening y anti-abuso

- CSP 'whitelist-only' sin inline; COOP/COEP; Referrer-Policy:no-referrer; HSTS.
- Rate-limit por IP/huella y tamaño máximo de secreto/archivo.
- Prueba de trabajo (Hashcash simple) al crear secretos públicos.

Día 6 – Pruebas, métricas y runbook de incidentes

- Pruebas cripto (IV/tag incorrectos deben fallar), e2e (creación→lectura→borrado).
- Métricas sin PII: conteos agregados, tiempos, falsas alarmas canary.
- Runbook: 'apertura sospechosa' → aislar, notificar, revocar, revisar auditoría.

Día 7 – Demo, README y despliegue

- Script de demo en 5 minutos, GIFs/imagenes y README con arquitectura.
- Despliegue en VPS/servicio IaaS educativo con TLS y variables seguras.
- Checklist de verificación (CSP válida, HSTS, logs sin PII, backups funcionando).

Backlog (épicas, historias y criterios de aceptación)

Épica 1 – Secreto de un solo uso

- Como usuario, quiero crear un secreto que se autodestruya tras su primera lectura.
- Criterio: segunda petición devuelve 410; el secreto jamás se almacena en claro.

Épica 2 – Cifrado cliente

- Como usuario, deseo que el cifrado ocurra en mi navegador y que el servidor no pueda leer mi secreto.
- Criterio: auditoría del código confirma uso de WebCrypto y ausencia de envío de K.

Épica 3 – Transparencia/auditoría

- Como auditor, deseo exportar un log encadenado verificable.
- Criterio: herramienta de verificación recalcula hash raíz con éxito.

Épica 4 – Detección (canary)

- Como propietario, quiero alertas si el secreto se abre en contexto sospechoso.
- Criterio: evento canario visible con sello de tiempo y etiqueta de riesgo.

Épica 5 – Hardening

- Como responsable de seguridad, quiero políticas de navegador y límites anti-abuso.
- Criterio: CSP válida, HSTS activo, rate-limit y PoW verificados.

Roles y responsabilidades (4 integrantes)

Rol A – Cripto & Cliente: WebCrypto (AES-GCM, HKDF), ECDH X25519 (o passphrase + Argon2id), manejo de #key, UI de creación/lectura, QR, pruebas de vectores.

Rol B – Backend & Políticas: API REST, semántica one-shot, expiración, PoW, rate-limit, borrado criptográfico y limpieza programada.

Rol C – Transparencia & Detección: audit_log encadenado, export/verify, panel de métricas sin PII, canary tokens y alertas.

Rol D – Hardening & Operaciones: headers (CSP/HSTS/etc.), CI, contenedor, backup cifrado y runbook de incidentes.

Estimación de plazos

Ruta rápida (1 semana):

- Días 1–2: Épicas 1–2 (MVP de creación/lectura E2E).
- Días 3–4: Épicas 3–4 (auditoría + canary).
- Día 5: Épica 5 (hardening + anti-abuso).
- Días 6–7: Pruebas, demo y despliegue.

Ruta cómoda (2 semanas): priorizar MVP en semana 1 y extras wow en semana 2.

Pruebas y script de demo (5 min)

- Crear secreto (UI indica 'cifrado local'). Copiar URL con #key → QR.
- Abrir en otro navegador → lectura única → backend marca burned.
- Intentar recargar → 410 Gone (captura en vivo).
- Abrir con VPN/incógnito → salta evento canario en panel.
- Forzar 'Destruir ahora' → intentar descifrar: falla verificada.
- Exportar log → recalcular hash raíz con script y mostrar OK.

Videos y recursos recomendados (para estudio y referencia)

- Onetime Secret (open-source y servicio): <https://github.com/onetimesecret/onetimesecret> · <https://onetimesecret.com/en/>
- Otra plataforma OSS similar (Luzifer/ots): <https://github.com/Luzifer/ots>
- Web Crypto API (MDN): https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API

- AES-GCM en WebCrypto (ejemplos):
<https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto/encrypt> y
<https://github.com/kbagher/aes-gcm>
- ECDH X25519 / libsodium (docs): https://libsodium.gitbook.io/doc/key_exchange
- Q/A y discusión técnica: <https://crypto.stackexchange.com/questions/81411>
- Shamir Secret Sharing (videos): <https://www.youtube.com/watch?v=v6lvxnJHD-o> ·
<https://www.youtube.com/watch?v=iFY5SyY3IMQ>
- Canary tokens (videos): <https://www.youtube.com/watch?v=14YuyMkfB9Q> ·
<https://www.youtube.com/watch?v=apixhc43JuE>

Snippets de referencia (pseudocódigo)

```
// AES-GCM en navegador (simplificado)
const iv = crypto.getRandomValues(new Uint8Array(12));
const key = await crypto.subtle.importKey('raw', masterKeyBytes, 'AES-GCM', false, ['encrypt', 'decrypt']);
const ct = await crypto.subtle.encrypt({name: 'AES-GCM', iv}, key, new TextEncoder().encode(secret));

// URL con fragmento (no enviado al servidor)
const url = origin + '/s/' + id + '#key=' + base64url(keyBytes);

// Verificación de log encadenado
for (const e of events) {
  e.hash = H( e.prev_hash + serialize(e.event) );
}
assert(chainIsValid(events));
```

Riesgos y mitigaciones

- Filtración del enlace (#key): usar passphrase adicional y expiraciones cortas; educar al usuario en UI.
- Phishing del dominio: HSTS, mostrar huella pública, preferir copiar enlace desde la app.
- Abuso/DoS: PoW + rate-limit + tamaño máximo de payload.
- Pérdida de clave: dejar claro que no hay recuperación (ética de zero-knowledge).

Apéndice – Checklist de verificación final

- TLS activo (no warnings), HSTS, CSP sin 'unsafe-inline', SRI en recursos.
- Lectura única comprobada; reintento devuelve 410.
- Export/verify del audit_log funciona (hash raíz coincide).
- Canary genera alerta y no guarda PII sensible.
- Backups cifrados de metadatos; restore probado.
- README con diagrama, decisiones de diseño y límites éticos.

Runbook de Incidentes (resumen)

- Detección: evento canario/sospechoso en panel.
- Contención: pausar creación pública, subir dificultad PoW, activar rate-limit estricto.
- Erradicación: revocar tokens/llaves afectadas, limpiar secretos pendientes.
- Recuperación: verificar integridad del audit_log, reactivar creación con monitoreo.
- Lecciones: actualizar amenazas conocidas, ajustar umbrales y documentación.